A readable HND/degree level textbook

Relational databases
B Eaglestone
Stanley Thornes (1991) 256 pp £12.99
ISBN 0 7487 1176 7

It is very difficult to recommend a good introductory database book for the HND/degree level. There are many books by authors such as Date and Elmasri which are excellent if you already have database experience, but these often go into too much depth and are aimed at proficient users rather than beginners. This book sets out to fill the gap in the market.

The book is written primarily for HND and second year degree students and it adequately covers the topics that are typically taught at this stage. Too many introductory books are lacking in some respect; often they do not cover all the topics, have become out of date, or are still too advanced for beginners.

Given the level the book is aimed at, there were initial reservations when looking at the list of topics—some of which are normally taught at final year degree level: for example, fourth and fifth normal form, distributed databases and relational calculus. These are, however, described in plain English, which most students will understand: it is a book they can still use once they move on to more advanced material.

The book aims to present the reader with an understanding of the ideas underlying relational technology, the programs and techniques that are used to apply the technology, which Eaglestone covers quite adequately. He sets out to answer the questions: How is a relational database used and designed? What housekeeping tasks are necessary? etc.

The first chapter gives an overview of the database approach. A case study based on a Sales Order processing system is introduced, and used throughout the book for examples. The next chapter looks at the relational model and relational database management systems (RDBMS). Relational algebra is introduced in simple terms, with plenty of examples. Many database books use mathematical symbols for relational algebra which is difficult for novices, or those without a mathematical background. This book is refreshing in that it gives plenty of understandable examples.

The third chapter looks at how a relational database is used, where it fits in the software development lifecycle, and reviews the range of tools for developing database applications, such as Fourth Generation Languages (4GLs). The INGRES 4GL is used as an example: SQL is discussed in great depth, again with plenty of examples; and embedded SQL is also shown to give a flavour of how it is used. For comparison, a few examples of QUEL and the equivalent relational calculus expressions are given to show alternative ways of expressing database operations.

Relational database design is the next subject, a topic most beginners find extremely difficult. The author presents both a top-down and bottom-up approach for data analysis, leaving it to the user to decide which method they prefer; both are described in a step-by-step fashion, which is easy to follow. The transformation of the conceptual model to the logical is clearly described. He explains why normalization is important, which would surprise some students who would have you believe that it is just something we make them do in order to pass their assessments.

Later chapters of the book cover housekeeping tasks, such as transaction handling, recovery and security, and distributed databases. Finally the author looks at the future, discussing the weaknesses of the relational model and what the next generation of database management systems may hold.

Eaglestone writes in a clear manner, with appropriate explanations well supported by plenty of examples. The author exhibits a wealth of experience of teaching at this level, showing a good insight into the degree of understanding by students new to the topic. On the other hand, it is not really a book for advanced database students who would need more depth and explanation of the subject.

Outside academia, users contemplating buying and using a relational database system might find the book useful; it could help the reader appraise and make effective use of relational databases. It is written in a style clear enough for non-database specialists to understand, although some knowledge of computing terms is assumed.

It is a very readable book, it fulfils its aim of being an HND/introductory degree level textbook, and at the price it offers good value for money.

M GARVEY
University of Wolverhampton, UK

Oases of sound, sensible lore for Objectory users

Object-oriented software engineering: a use case driven approach
I Jacobson, M Christerson, P Jonsson and G Övergaard
Addison-Wesley (1992) £29.95 524 pp
ISBN 0 201 54435 0

It is a useful rule of thumb always to be chary of any book whose title contains either the words 'object-oriented' or the words 'software engineering', so it was with some trepidation that I approached this volume. The fact that I was unable to parse the phrase 'a use case driven approach' in any way that had meaning only made matters worse. Luckily my caution was unnecessary. The only danger I was in from this worthy tome was of nodding off and dropping it, hard covers and all, on to my toes. Which is a shame really because it contains oases of sound, sensible lore amongst much arid pseudo-science.

The generality of the title is a blind. The book is actually a sales pitch for the ideas behind Objectory, a system that the blurb claims is 'the first commercially available comprehensive object-oriented process for developing large-scale industrial systems'. I suspect others would challenge this claim vociferously, and it made me instantly aware that what we have here is snake oil. This particular snake oil may be a genuine panacea and live up to all its claims, but it is still snake oil with all that that entails for the unwary reader. The authors do devote the last chapter of the book to a comparison of their remedy with other popular and well-known brands, but sadly, this is altogether too polite. A little more odious comparison would have made for better reading and would have told me more. If you run a medicine show then run it properly.

But what of the content? Well, there is a lot of it. All logically structured and coherently arranged, covering everything from the basics of object-orientation, through analysis, testing to project management. All this, we must remember, strongly scented with Objectory. Unfortunately I disagreed with the ideas on the very first page, baulked at the definition

of 'object' proposed by the authors, and became utterly bemused by the many box-and-arrow diagrams. Some careful cutting and reshaping of the text would make it much more attractive and genuinely informative. I particularly liked the section on testing, and the cases studied have considerable potential, but a lot would have to go before I felt happy with the overall 'Objectory-oriented' approach. And those diagrams! The person who invented graphics packages that authors could use has much to answer for. So has the person who invented electronic clip art as there are some fairly dire examples of that included as well. The only positive thing that can be said about the illustrations is that the authors do follow their precepts and reuse some of them several times.

What, then, is the audience for this book? If you are a user of Objectory, this book is a bible. If you are what the preface calls 'an experienced software engineer' (I confess that I have never met such a beast) but are from another school then this book will show you a different viewpoint but will not tell you much that is new. Novices could learn from this book, except that they would get a blinkered, or at least, a narrow view of the world. If you are a manager then this is a 'must read'. It will fulfil all your fantasies of a well-controlled world where dependable software is delivered on time and on spec by organized, creative, well-motivated programmers and more. While you are ploughing through the text your employees can get on with some real work without you breathing down their necks.

As you will have gathered, I did not like this book. Not because it is a bad book, not even because it is dull, but because it presents an approach to the development of software which I think is in many respects wrong. I am not so certain that we know, or indeed will ever know, enough about the design and construction of software that we can adopt such methodological approaches as advocated in this book. Other, longer-established fields of endeavour have tried and abandoned design methodologies as unworkable. The raw material used in the construction of this book is good, but the glue that holds it together is weak.

L F MARSHALL
Department of Computing Science,
University of Newcastle upon Tyne,
UK

A book for the project manager's library

Managing computer projects
R Gibson
Prentice Hall International (UK)
(1992) 161 pp £22.95
ISBN 0 13 159591 1

Nowadays projects tend to be more and more complex and obviously the risk for a disaster (overtime, overbudget, or project cancellation) becomes greater. Usually it is too difficult to identify the reason(s) or culpable(s) for this disaster, therefore responsibilities decrease.

How can an organization finish a project on time and on budget? The best answer to this question is: You must have an appropriate management, in connection with a proper manager. So, the main topic covered by this book is the role of the project manager within the organization of development and his relation with the purchaser.

After a short and precise introduction, the largest part of the book (Chapters 2–5) covers the software life-cycle (the project launching, analysis, design, planning, implementing and the acceptance), showing the manager's main goal. The last chapter deals with the manager himself (nature of work, selection methods, skills profile, etc.). One of the significant subjects throughout the book is the importance of the appointment of a manager at the early stages of the project.

As the author states in the preface, this book is directed to skilled staff, but it also addresses two types of reader: On one side, the senior staff that belongs to an organization and that is responsible for software development (contractor). It includes the management upper level, showing the consequences for erroneous decisions and outlining some solutions. It is here where the author proves his own background on software management. There are no quantitative results (with the exception of the discussion about error removal in Chapter 5), because these would be the topic for another book on planning or metrics, and that is not the purpose. On the other side, the book is also designed for the purchaser, due to its qualitative aspect. The development team must be connected to the purchaser's staff (especially the project manager). So another relevant matter of the book is the co-operation between the development team and the purchaser, in order to obtain the desired project with an extraordinary quality.

The need for a satisfactory Software Quality Assurance process is not looked at as an independent organization. On the contrary, it is included in the role of the software manager, justifying the necessity for a rigorous quality control with emphasis in software configuration control.

The author's style is concise. The book is well structured and rich in introductions, enumerations of learned lessons, conclusions and outlines. There is a selected collection of brief case studies that reveals some common errors made in software management. Although some of them are rather extreme situations, they are useful for the understanding of the main chapters of the book. Learning from past mistakes is one of the techniques used by the writer in the book.

Taking into consideration the development experience that the author reflects on his pages, this work is not exactly expensive. This, combined with the helpful thoughts that can be taken from it, makes this book a significant contribution to our library.

J TUYA
University of Oviedo, Spain

Clear and comprehensive presentation of DLP

DLP: a language for distributed logic programming design, semantics and implementation
A Eliens
John Wiley (1992) 325 pp £39.95
ISBN 0 471 93117 9

This book presents a particular approach to the problem of combining three modern programming paradigms —the paradigms of logic programming, object-oriented programming and distributed information processing. In recent years we have seen intensive research efforts for combinations of these paradigms, usually in pairs. These efforts are all interesting and practically useful.

The book deals with the following main topics: The design of the language DLP (Distributed Logic Programming) is described in the first part. It is developed on the base of appropriate extensions of Prolog, including such notions as objects, inheritance, dynamic creation of objects, message passing, and concurrency. An interesting question in the